Parallel simulations

Simulation studies — parameter recovery, scenario sweeps, prior sensitivity — are loops of *independent* fits. Written as a plain `for` loop they run one at a time, so a single core works while the rest of the machine sits idle.

`run_simulations` spreads that loop across worker processes. Each worker is a separate Python process with its own JAX backend, so every simulation gets its own `bf()` instance, its own share of the cores and its own seed.

`run_simulations` is part of the Python package. In R and Julia, the same pattern is written with the parallel tools of the language, `parallel::parLapply()` and `Distributed.pmap()`: see the R and Julia tabs. Each worker is again a separate process with its own `importBF()` session and seed.

Crucially, **the scenario settings travel with the results**: every row of the returned table carries the parameters that produced it, so comparing true against estimated values needs no bookkeeping of your own.

---

1. The pattern

Write a function that runs **one** simulation and returns the numbers you care about. `run_simulations` calls it once per scenario, in parallel, and stacks the results into a `pandas` DataFrame.

Python

```
from BayesForge import bf, grid, run_simulations

def one_sim(<your scenario parameters>, seed, **_):
    m = bf(rand_seed=seed, print_devices_found=False)  # one bf per simulation
```

```
    data = <simulate a dataset using the scenario parameters>
    m.fit(<model>, obs=data, progress_bar=False)

    return {"estimate": ..., "truth": ...}   # -> one row of the result table

scenarios = grid(<param>=[...], <param>=[...], reps=<n>)   # or just an int
res = run_simulations(one_sim, scenarios)                  # -> DataFrame
```

R

```
library(parallel)

one_sim <- function(<your scenario parameters>, seed) {
    library(BayesForge)                    # each worker is a fresh R session
    m <- importBF(rand_seed = seed, cores = 1, print_devices_found = FALSE)

    data <- <simulate a dataset using the scenario parameters>
    m$fit(<model>, obs = data, progress_bar = FALSE)

    data.frame(estimate = ..., truth = ...)   # -> one row of the result table
}

scenarios <- expand.grid(<param> = c(...), <param> = c(...), rep = 1:<n>)
scenarios$seed <- seq_len(nrow(scenarios))   # one seed per simulation

cl <- makeCluster(<workers>)
rows <- parLapply(cl, seq_len(nrow(scenarios)), function(i, sc, f) {
    cbind(sc[i, ], do.call(f, sc[i, setdiff(names(sc), "rep")]))
}, scenarios, one_sim)
stopCluster(cl)
res <- do.call(rbind, rows)                  # -> data.frame
```

Julia

```
using Distributed
addprocs(<workers>; exeflags = "--project=$(Base.active_project())")
@everywhere using PyBayesForge

@everywhere function one_sim(<your scenario parameters>, seed)
```

```
    m = importBF(rand_seed = seed, cores = 1, print_devices_found = false)

    data = <simulate a dataset using the scenario parameters>
    m.fit(<model>, obs = data, progress_bar = false)

    (estimate = ..., truth = ...)          # -> one row of the result table
end

scenarios = [(<param> = a, <param> = b, rep = r, seed = i) for (i, (a, b, r)) in
             enumerate(Iterators.product([...], [...], 1:<n>))]
rows = pmap(sc -> merge(sc, one_sim(sc.<param>, sc.<param>, sc.seed)), scenarios)
```

Three things the runner injects into every call, on top of your own parameters:

injected	meaning
`sim`	global index of the simulation
`rep`	replicate index within its scenario cell
`seed`	per-simulation seed, derived deterministically from the base seed

Pass `seed` into `bf(rand_seed=seed)`. Without it every worker draws the *same* data, because `bf()` seeds the global NumPy and Python RNGs from that value.

Always give `one_sim` a `**_` catch-all so extra injected keys cannot break it.

> **!** In a `.py` script, use a main guard
>
> Workers are *spawned*, which re-imports the calling script. Without the guard the module level runs again in every worker:
>
> **Python**
>
> ```
> if __name__ == "__main__":
>     res = run_simulations(one_sim, scenarios)
> ```
>
> **R**

```r
# PSOCK workers are new R sessions that do not run your script:
# load BayesForge inside one_sim (or with clusterEvalQ) and stop the cluster.
cl <- makeCluster(4)
clusterEvalQ(cl, library(BayesForge))
on.exit(stopCluster(cl), add = TRUE)
```

Julia

```julia
# Workers are new Julia processes: everything they run must be defined
# @everywhere, after addprocs().
addprocs(4; exeflags = "--project=$(Base.active_project())")
@everywhere using PyBayesForge
```

Notebooks and the REPL need no guard.

Return numbers, not the model

A fitted `bf` object cannot cross a process boundary — `m.diag` holds closures over `m`, and `m.dist` is an unpicklable proxy. Return `m.summary()`, `m.sampler_stats()`, or a plain dict. Returning `m` raises an explicit error.

---

2. A concrete example

Recovering the slope of a linear regression across two sample sizes and three true slopes, four replicates each — 24 simulations.

Python

```python
import numpy as np
from BayesForge import bf, grid, run_simulations

def one_sim(N, b_true, seed, **_):
    m = bf(rand_seed=seed, print_devices_found=False)

    # --- simulate one dataset ---------------------------------------
    x = np.asarray(m.dist.normal(0.0, 1.0, sample=True, shape=(N,)))
```

```
    y = np.asarray(m.dist.normal(1.5 + b_true * x, 1.0, sample=True))

    # --- the model ------------------------------------------------
    def linreg(x=None, y=None):
        a = m.dist.normal(0.0, 5.0, name="a")
        b = m.dist.normal(0.0, 5.0, name="b")
        s = m.dist.exponential(1.0, name="s")
        m.dist.normal(a + b * x, s, name="y", obs=y)

    # --- fit, then report truth against estimate ------------------
    m.fit(linreg, obs=dict(x=x, y=y),
          num_warmup=500, num_samples=500, num_chains=1,
          progress_bar=False)

    est = m.summary()
    return {"b_hat":  float(est.loc["b", "mean"]),
            "b_true": b_true,
            "r_hat":  float(est.loc["b", "r_hat"])}

if __name__ == "__main__":
    scenarios = grid(N=[100, 500], b_true=[-0.8, 0.0, 0.8], reps=4)   # 24 sims
    res = run_simulations(one_sim, scenarios)

    print(res[["sim", "N", "b_true", "b_hat", "elapsed_s"]].head())

    # Mean absolute error of the slope per scenario
    print((res.b_hat - res.b_true).abs().groupby([res.N, res.b_true]).mean())
```

R

```
library(parallel)

one_sim <- function(N, b_true, seed) {
    library(BayesForge)
    m <- importBF(rand_seed = seed, cores = 1, print_devices_found = FALSE)

    # --- simulate one dataset -------------------------------------
    x <- bf.dist.normal(0.0, 1.0, sample = TRUE, shape = N)
    y <- bf.dist.normal(1.5 + b_true * x, 1.0, sample = TRUE)

    # --- the model ------------------------------------------------
```

```r
    linreg <- function(x, y) {
        a <- bf.dist.normal(0.0, 5.0, name = "a")
        b <- bf.dist.normal(0.0, 5.0, name = "b")
        s <- bf.dist.exponential(1.0, name = "s")
        bf.dist.normal(a + b * x, s, name = "y", obs = y)
    }

    # --- fit, then report truth against estimate -------------------
    m$fit(linreg, obs = list(x = x, y = y),
          num_warmup = 500L, num_samples = 500L, num_chains = 1L,
          progress_bar = FALSE)

    est <- m$summary()
    data.frame(b_hat  = est["b", "mean"],
               b_true = b_true,
               r_hat  = est["b", "r_hat"])
}

scenarios <- expand.grid(N = c(100, 500), b_true = c(-0.8, 0.0, 0.8), rep = 1:4)  # 24 sims
scenarios$seed <- seq_len(nrow(scenarios))

cl <- makeCluster(4)                          # e.g. detectCores() workers
rows <- parLapply(cl, seq_len(nrow(scenarios)), function(i, sc, f) {
    cbind(sim = i, sc[i, ], f(sc$N[i], sc$b_true[i], sc$seed[i]))
}, scenarios, one_sim)
stopCluster(cl)
res <- do.call(rbind, rows)

print(head(res[, c("sim", "N", "b_true", "b_hat")]))

# Mean absolute error of the slope per scenario
print(aggregate(abs(b_hat - b_true) ~ N + b_true, data = res, FUN = mean))
```

Julia

```julia
using Distributed
addprocs(4; exeflags = "--project=$(Base.active_project())")  # e.g. Sys.CPU_THREADS workers
@everywhere using PyBayesForge, PythonCall

@everywhere function one_sim(N, b_true, seed)
    m = importBF(rand_seed = seed, cores = 1, print_devices_found = false)
```

```
    # --- simulate one dataset ------------------------------------
    x = m.dist.normal(0.0, 1.0, sample = true, shape = (N,))
    y = m.dist.normal(1.5 + b_true * x, 1.0, sample = true)

    # --- the model -----------------------------------------------
    @BF function linreg(x, y)
        a = m.dist.normal(0.0, 5.0, name = "a")
        b = m.dist.normal(0.0, 5.0, name = "b")
        s = m.dist.exponential(1.0, name = "s")
        m.dist.normal(a + b * x, s, name = "y", obs = y)
    end

    # --- fit, then report truth against estimate -----------------
    m.fit(linreg, obs = pydict(x = x, y = y),
          num_warmup = 500, num_samples = 500, num_chains = 1,
          progress_bar = false)

    est = m.summary()
    (b_hat  = pyconvert(Float64, est.loc["b", "mean"]),
     b_true = b_true,
     r_hat  = pyconvert(Float64, est.loc["b", "r_hat"]))
end

scenarios = [(sim = i, N = N, b_true = b, rep = r, seed = i) for (i, (N, b, r)) in
             enumerate(Iterators.product([100, 500], [-0.8, 0.0, 0.8], 1:4))]  # 24 sims
res = pmap(sc -> merge(sc, one_sim(sc.N, sc.b_true, sc.seed)), scenarios)

using DataFrames
res = DataFrame(res)
println(first(res[:, [:sim, :N, :b_true, :b_hat]], 5))

# Mean absolute error of the slope per scenario
println(combine(groupby(res, [:N, :b_true]), [:b_hat, :b_true] => ((h, t) -> sum(abs.(h .- t)
```

`res` is a tidy DataFrame — the grid columns `N` and `b_true` are joined onto every row automatically:

```
   sim    N  b_true  b_hat  elapsed_s
0    0  100    -0.8  -0.64      8.952
1    1  100    -0.8  -0.80      9.099
2    2  100    -0.8  -0.93      8.598
```

```
3    3  100    -0.8  -0.62      8.620
4    4  100     0.0  -0.13      8.184
```

So the analysis is a `groupby`, with nothing to join by hand (the last line of the script above):

Python

```python
(res.b_hat - res.b_true).abs().groupby([res.N, res.b_true]).mean()
```

R

```r
aggregate(abs(b_hat - b_true) ~ N + b_true, data = res, FUN = mean)
```

Julia

```julia
combine(groupby(res, [:N, :b_true]), [:b_hat, :b_true] => ((h, t) -> sum(abs.(h .- t)) / leng
```

```
N    b_true
100  -0.8      0.118
      0.0      0.073
      0.8      0.055
500  -0.8      0.015
      0.0      0.025
      0.8      0.018
```

Alongside your own columns, every row carries `sim`, `rep`, `seed`, `worker`, `elapsed_s` and `error`.

---

3. Chains, workers and cores

Chains and simulations compete for the same cores, so `num_chains`, `workers` and the fit's `chain_method` must be chosen together. There are three sensible setups. All produce the same number of genuine chains and equivalent ESS — they differ in speed and in how many simulations are in flight at once.

fit's `chain_method`	pass `num_chains=`	workers	cores/worker	sims at once
(single chain)	omit	`n_cpu`	1	`n_cpu`
`'parallel'`	`N`	`n_cpu // N`	`N`	`n_cpu // N`
`'vectorized'`	omit	`n_cpu`	1	`n_cpu`

Single-chain fits are the default and the quickest way through a large batch when you do not need a per-fit R-hat.

`chain_method='parallel'` puts one chain on one JAX device, so it needs a device per chain. Pass `num_chains` to the runner and take it back in your function, so the core budget and the fit can never disagree:

Python

```python
def one_sim(N, seed, num_chains, **_):
    ...
    m.fit(model, obs=data, num_chains=num_chains,
          chain_method="parallel", progress_bar=False)

res = run_simulations(one_sim, scenarios, num_chains=4)
```

R

```r
one_sim <- function(N, seed, num_chains = 4) {
    library(BayesForge)
    # one JAX device per chain: give the worker as many cores as chains
    m <- importBF(rand_seed = seed, cores = num_chains, print_devices_found = FALSE)
    ...
    m$fit(model, obs = data, num_chains = as.integer(num_chains),
          chain_method = "parallel", progress_bar = FALSE)
}
```

```
cl <- makeCluster(parallel::detectCores() %/% 4)    # workers x 4 cores
```

Julia

```
@everywhere function one_sim(N, seed; num_chains = 4)
    # one JAX device per chain: give the worker as many cores as chains
    m = importBF(rand_seed = seed, cores = num_chains, print_devices_found = false)
    ...
    m.fit(model, obs = data, num_chains = num_chains,
          chain_method = "parallel", progress_bar = false)
end

addprocs(Sys.CPU_THREADS ÷ 4; exeflags = "--project=$(Base.active_project())")  # workers x 4
```

On a 24-core machine `num_chains=4` gives each worker 4 cores and 4 JAX devices, pins it to its own block of CPUs, and runs **6 simulations at a time** instead of 24 — simulation concurrency traded for chain concurrency.

`chain_method='vectorized'` vmaps the chains inside a *single* device, so no cores are reserved for them and every core stays free for a different simulation. Do **not** pass `num_chains` for this one: it would reserve cores that vectorized chains never use.

Measured on 24 simulations over 24 cores, each drawing 4 chains at equal ESS:

setup	wall
`workers=24`, `cores_per_worker=1`, vectorized	**17.5 s**
`num_chains=4` (6 workers × 4 cores), parallel	21.9 s
`workers=6`, `cores_per_worker=4`, vectorized	29.1 s

The silent trap

Asking a fit for `num_chains=4` with `chain_method='parallel'` *without* telling the runner leaves the worker with one device. NumPyro then draws the chains **sequentially** — correct results, needlessly slow, with a warning per fit. Passing `num_chains` to `run_simulations` prevents it.

4. Useful options

argument	what it does
`workers`	How many simulations run at once. Defaults to one per core, divided by `cores_per_worker`.
`fn_kwargs`	Constants passed to every call. Serialised **once per worker** rather than once per scenario — put large shared arrays here.
`seed`	Base seed. Per-simulation seeds derive from it deterministically, so a rerun reproduces regardless of worker count or completion order.
`on_error`	`'record'` (default) puts the traceback in the row's `error` column and keeps going; `'raise'` aborts the run.
`backend`	`'serial'` runs in-process for debugging, so `pdb` and full tracebacks work.

Inspecting failures without losing the rest of the batch:

Python

```python
res = run_simulations(one_sim, scenarios, on_error="record")
res[res.error.notna()][["sim", "N", "error"]]
```

R

```r
# Record the error of a failed simulation instead of losing the whole batch
safe_sim <- function(...) tryCatch(cbind(one_sim(...), error = NA_character_),
                                   error = function(e) data.frame(error = conditionMessage(e)
```

Julia

```
# Record the error of a failed simulation instead of losing the whole batch
@everywhere safe_sim(args...) = try
    merge(one_sim(args...), (error = missing,))
catch e
    (error = sprint(showerror, e),)
end
```

JIT Caching: Define the model at module level for maximum speed

Each worker JIT-compiles the model on its first fit. When the model is built fresh inside `one_sim` (a closure, as in the example above), *every* simulation pays full compilation because the JIT cache key changes each time. Defining it at module level lets a worker reuse its compiled executable across all the simulations it handles — often the single largest speedup available.
To do this, ensure your global model function accepts `m` (the simulation's specific `bf` instance) as an argument, and wrap it in a `lambda` when calling `m.fit`:

Python

```
# 1. Define the model globally, requiring 'm' as an argument
def linreg(m, x=None, y=None):
    a = m.dist.normal(0.0, 5.0, name="a")
    b = m.dist.normal(0.0, 5.0, name="b")
    s = m.dist.exponential(1.0, name="s")
    m.dist.normal(a + b * x, s, name="y", obs=y)

def one_sim(N, b_true, seed, **_):
    m = bf(rand_seed=seed, print_devices_found=False)

    # [ simulate x and y here ]

    # 2. Use a lambda to inject 'm' into the global function
    m.fit(lambda x=x, y=y: linreg(m, x, y),
          obs=dict(x=x, y=y),
          num_warmup=500, num_samples=500, num_chains=1,
          progress_bar=False)

    return {"b_hat": float(m.summary().loc["b", "mean"])}
```

R

```r
# R: m$fit() converts the R model into a new Python function at every call,
# so the compiled model is not reused across fits; define the model once
# for readability only.
linreg <- function(x, y) {
    a <- bf.dist.normal(0.0, 5.0, name = "a")
    b <- bf.dist.normal(0.0, 5.0, name = "b")
    s <- bf.dist.exponential(1.0, name = "s")
    bf.dist.normal(a + b * x, s, name = "y", obs = y)
}
```

Julia

```julia
# Julia: m.fit() converts the @BF model into a new Python function at every
# call, so the compiled model is not reused across fits; define the model
# once for readability only.
@everywhere linreg_for(m) = begin
    @BF function linreg(x, y)
        a = m.dist.normal(0.0, 5.0, name = "a")
        b = m.dist.normal(0.0, 5.0, name = "b")
        s = m.dist.exponential(1.0, name = "s")
        m.dist.normal(a + b * x, s, name = "y", obs = y)
    end
    linreg
end
```